TraceLens: Multimodal Relative Debugging of UI Test Regressions via Pass–Fail Trace Comparison

Musab Ahmed Khan*
Sabanci University
Istanbul, Turkey
musab.khan@sabanciuniv.edu

ABSTRACT

When an automated UI test regresses from pass to fail, engineers practice *relative debugging*: they compare two executions of the same test to find the earliest step whose divergence explains the failure, separating root cause from downstream symptoms. We present TraceLens, a multimodal pipeline that aligns pass and fail traces, scores per-step *relative anomalies* across network logs, console output, actions, navigation, and screenshots, and optionally reranks candidates with a large language model (LLM) and a vision-language model (VLM). Our first architecture (v1) stacked heuristic ranking, LLM reranking, VLM ensemble, and trace-specific Hit@1 guards; hybrid mode proved brittle when guards tuned for one failure pattern regressed others. We therefore introduce v2, which replaces post-hoc overrides with *unified weighted fusion* and read-only diagnosis. On a benchmark of 22 real UI regression traces, a heuristic baseline achieves 64% Hit@1. LLM-only reranking reaches **86% Hit@1**, the best top-1 localization. Hybrid LLM+VLM fusion reaches **82% Hit@1** but **100% Hit@3** with lower mean rank distance (0.27 vs. 0.32), recovering visually dominant cases while occasionally regressing when visual noise overrides causal text. We recommend LLM-only for sharp top-1 localization and hybrid fusion when top-k robustness and screenshot evidence matter.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → *Machine learning*.

KEYWORDS

relative debugging, regression diagnosis, UI testing, execution traces, fault localization, LLM, VLM

1 INTRODUCTION

Automated end-to-end (E2E) UI tests produce rich *execution traces*: ordered steps recording browser actions, HTTP traffic, console messages, and screenshots. When a test that passed yesterday fails today, debugging is inherently *relative*: engineers diff the good run against the bad run to locate where behavior first diverged. We call this task **relative debugging**, following the compare-against-reference paradigm of Abramson et al. [1, 2, 10]: given a passing trace P and a failing trace F for the same test case, rank execution steps by how likely each is the *earliest root cause* of the regression.

Relative debugging differs from classical source-level fault localization [3, 12]. The artifact under analysis is an *execution trace*, not a program graph. The supervision signal is the pass–fail *pair* itself: anomalies are defined relative to what changed between runs.

Three difficulties recur in practice. **(1) Symptom vs. cause.** A wrong credential at step 7 may surface only as a verification failure at step 8; ranking by error magnitude favors the symptom [7]. **(2) Silent and visual faults.** Missing form fields, CAPTCHA walls, and stale CDN assets may produce no console or network signal; only screenshots reveal the divergence [8]. **(3) Noise.** Telemetry URLs, placeholder domains, and transient network flicker create spurious diffs that distract both heuristics and language models.

We built TraceLens to automate relative debugging for UI test traces. The system parses heterogeneous trace formats, aligns pass and fail steps, extracts multimodal relative features, ranks suspicious steps, and generates technical and stakeholder-facing explanations. We evaluate five configurations against a **heuristic baseline** (pass–fail signal diffing without API calls): a no-pixel ablation, LLM reranking, VLM screenshot analysis, and hybrid fusion combining LLM+VLM.

Running example. On the `hackernews` trace, a Firebase timeout at step 12 causes many quiet steps afterward; heuristics rank a later verify step with higher pixel score. The LLM reranker reads `errors_observed_on_next_step` and promotes step 12, a canonical *relative* causal pattern. On `amazon`, the ground-truth fault is an auth-gated click (step 20); text and visual channels can disagree on which step to rank first—we analyze such cases in Section 4.

Section 4 reports results for all five configurations on 22 traces and discusses which mode fits different debugging workflows; we do not assume a winner before measurement.

Contributions.

- **TraceLens**, an end-to-end relative debugging pipeline for UI regression traces with interpretable multimodal signals and stakeholder reports.
- **Multimodal relative features**, including navigation mismatch detection, visual-causal walk-back from persistent screenshot divergence, and observer vs. action attribution.
- **v2 unified fusion**, replacing v1's trace-specific guard stack with a single weighted ranker; LLM/VLM outputs are soft priors and diagnosis is read-only.
- **Empirical study** comparing all five configurations on 22 diverse traces, with ablations and per-mode analysis.

2 BACKGROUND AND RELATED WORK

Relative and delta debugging. Abramson et al. [2] introduced *relative debugging*: locating errors by comparing a suspect program's state against a trusted reference execution. Their SC'95 application to large numerical models [1] validated the approach on evolving scientific codes. The Guard tool [10] operationalized relative debugging for software evolution. We adapt the same principle to

*CS 460 Project, Sabanci University. Student ID: 33017.

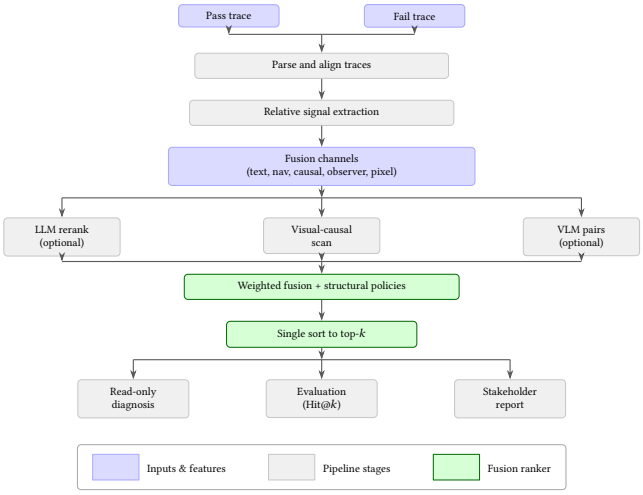

Figure 1: TraceLens v2 pipeline. Pass and fail traces are parsed, aligned, and differenced into fusion channels; optional LLM/VLM priors feed a single weighted sort. Diagnosis is read-only.

UI test traces: the pass run is the reference and the fail run is the suspect; divergence is measured per aligned step rather than per variable. Delta debugging [14] isolates failure-inducing *inputs* by systematically simplifying test cases; relative debugging instead ranks *execution steps* where pass and fail behavior diverge. Trace comparison also appears in log analysis [7, 13] and web testing [8, 9], but prior tools rarely publish reproducible ranking metrics over multimodal UI traces with ground-truth fault steps.

Fault localization. Spectrum-based and learning-based fault localization [11, 12] rank *source entities* (statements, methods) using coverage and pass/fail test outcomes. TraceLens instead ranks *trace steps* using direct pass–fail diffs of actions, logs, and pixels. The tasks are complementary: trace-step localization supports test maintainers; code-level FL supports developers patching production code.

LLMs and VLMs for software engineering. Recent surveys document LLM use for bug repair, log parsing, and code generation [4, 6]. We use an LLM as a *reranker* over compact relative features, not as a patch generator. Vision-language models enable GUI understanding [5, 15, 16]; we apply VLMs to *pass vs. fail screenshot pairs* at candidate steps identified by cheaper heuristics, controlling API cost.

Gap. No prior work combines (i) heterogeneous real UI trace schemas, (ii) explicit relative multimodal features, (iii) LLM and VLM priors in one fusion ranker, and (iv) Hit@k evaluation including text-invisible faults, in a reproducible open-source implementation with frozen benchmark runs.

3 APPROACH

Figure 1 summarizes our v2 architecture (entry point `main2.py`). Section 3.6 briefly describes v1, which we supersede.

Table 1: Pass–fail relative signals in TraceLens.

Signal	Relative comparison	Example
Network	New/changed errors in fail	HTTP 500
Console	New JS errors in fail	UI crash
Action	Text or type diverges	Invalid form
Navigation	Wrong page loaded	Wrong click
Pixel	Screenshot diff	Visual regression

3.1 Task formulation

Input: passing trace $P = (p_0, \ldots, p_{n-1})$ and failing trace $F = (f_0, \ldots, f_{m-1})$ for the same test.

Output: ranked step list $(s_1, \ldots, s_k)$, a technical diagnosis, and a plain-language summary.

Each step is unified into a schema with action text, action type, optional intent field (one data source), network logs, console logs, and screenshot path. Alignment uses index-based pairing when $|P| = |F|$; otherwise action-similarity alignment when faults insert or delete steps.

Ground truth is a single fault step index per trace (used only for evaluation). Metrics: Hit@k (GT in top-k), mean rank distance $|\text{rank}(GT) - 1|$, and MAD@5 (mean absolute deviation of GT rank from 1, capped at 5).

3.2 Relative signal extraction (heuristic baseline)

The **heuristic baseline** scores each aligned pair (p_i, f_i) without API calls. Four text signals in $[0, 1]$ are combined with fixed weights (renormalized when intent is absent):

- **Network:** new HTTP errors, status changes, or missing expected requests in F.
- **Console:** new JavaScript errors or warnings in F.
- **Action:** divergence in action text or type between P and F.
- **Intent:** changed verification outcome strings (available on one source schema).

Navigation signals detect wrong-page loads: expected URL absent and unintended page present (`wrong_navigation`), boosting steps that explain silent navigation faults.

Noise filtering removes placeholder domains (e.g., `example.com` distractors) and known telemetry patterns so relative diffs focus on test-relevant traffic.

Pixel diff (local, no API) compares pass/fail PNGs for the heuristic top-5; enabled by default, disabled with `-no-pixel`.

Table 1 summarizes signals; the baseline mode is `python main2.py` without flags.

3.3 Visual relative signals

Three visual layers differ in scope and cost:

Pixel boost computes mean pixel difference on aligned PNG pairs (local, cheap).

Visual-causal attribution scans all steps when visual mode is enabled, finds the earliest *persistent* screenshot divergence, and walks back to a silent cause step (e.g., missing zipcode visible only on the next screen). This uses deterministic rules, not a neural model.

VLM analysis sends pass/fail pairs for top-K candidates to a vision-language model, returning per-step visual scores and a suggested root step. Visual-causal steps are injected into the VLM candidate pool; scores can backfill from symptom steps onto cause steps.

3.4 LLM relative reranking

With `-llm`, a language model reranks a candidate pool: heuristic top-15 plus injected steps with strong navigation, causal, or action-changed signals. The prompt exposes compact *relative* fields (`action_changed`, `wrong_navigation`, filtered network/console snippets) without screenshots.

The LLM returns an ordered list and a diagnosis. In **v2**, rerank positions map to an `llm_prior` fusion channel; diagnosis **does not** change the final rank.

We use OpenRouter with temperature 0.0 (Nemotron-3-Super-120B for text) to maximize reproducibility; provider routing may still introduce variance.

Why top-15 for the LLM pool. Our traces contain roughly 20–40 aligned steps. Sending all steps to the LLM (`pre_llm_k=0`) increases token cost and context length on free-tier APIs without improving aggregate Hit@1 on our benchmark. Conversely, restricting the pool to heuristic top-5 would miss causal steps that score low on text alone (e.g., silent navigation faults later promoted by the LLM). **Top-15** balances recall and cost: it covers the heuristic shortlist while staying within prompt limits. Steps outside the top-15 can still enter the pool via *injection* when they carry strong relative signals (navigation mismatch, action change, errors on the next step, visual-causal root). This design mirrors test-information visualization work that highlights a focused subset of execution events rather than entire traces [7]. The value is fixed in `config.yaml` before evaluation.

Why top-5 for VLM screenshot pairs. VLM inference is the dominant API cost: with `per_step=true`, each analyzed step requires one multimodal call per trace. We set `top_k_for_vlm=5` to match the final ranked output size (`top_k=5`) and our Hit@5 metric, while keeping daily free-tier quotas feasible on 22 traces. Visual-causal divergence steps are injected into the VLM pool even when they fall outside the top-5 heuristic band, analogous to LLM injection. Section 5 notes that incomplete screenshot artifacts (e.g., `dictionary`) can prevent VLM analysis of the full top-5 pool.

3.5 v2 unified fusion

v2's main design contribution is a **single fusion score** per step:

$$S(i) = \sum_{c \in C} w_c \cdot f_c(i) \tag{1}$$

where channels C include text, navigation, causal action, observer symptom, visual causal, pixel, and optionally LLM and VLM priors (denoted `llm_prior` and `vlm_prior` in code). Weights w_c are fixed in configuration (`llm_prior` = 0.28, `vlm_prior` = 0.18, mirroring v1's 60/40 LLM/VLM blend).

Causal attribution distinguishes *action* steps (clicks, typing) from *observer* steps (verify, wait). When errors appear on a verify step, a preceding action step may be the true relative root (`causal_action` channel). When the verify step itself is labeled as

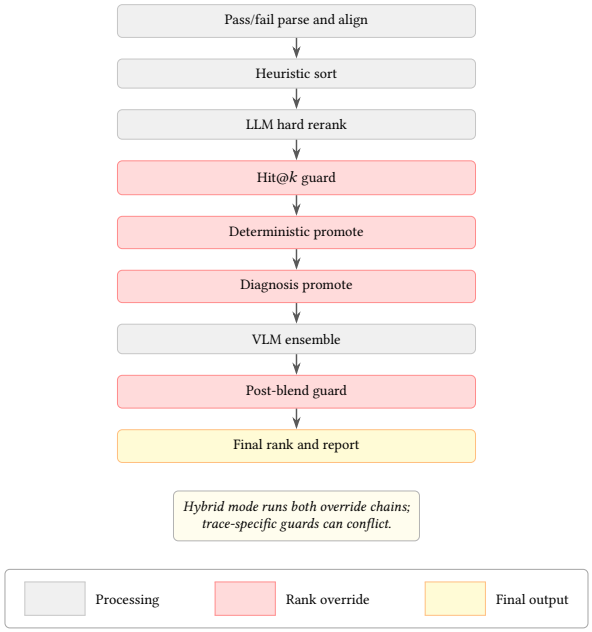

Figure 2: v1 stacked pipeline (`main.py`). Highlighted stages can override prior rankings. Hybrid runs LLM overrides before VLM overrides and a second guard pass.

ground truth (symptom-on-verify faults), observer channels must not be over-capped; v1 learned this through per-trace guards, v2 encodes it as channel weights and the observer-after-causal cap.

Structural policies (not trace-specific guards) adjust scores:

- **Observer cap:** limit verify-step visual weight when a prior causal action step has strong text support.
- **Downstream penalty:** demote steps after inferred root, with exemption when text signal exceeds a threshold.
- **LLM/action bonuses:** small nudges when the LLM ranks a causal or action-changed step first.

Steps are sorted once by $S(i)$. Hybrid mode (`-llm -vlm`) activates all channels; LLM-only and VLM-only use the same scorer with different channel subsets.

Evaluation modes. We compare five configurations from the same v2 ranker (Section 4 reports outcomes):

- **Heuristic (baseline):** pass–fail diff only; no LLM/VLM calls (`python main2.py`).
- **No pixel:** baseline without local screenshot diff (`-no-pixel`).
- **LLM:** adds text reranking and an `llm_prior` channel (`-llm`).
- **VLM:** adds screenshot-pair analysis and a `vlm_prior` channel (`-vlm`).
- **Hybrid:** all channels active (`-llm -vlm`).

3.6 v1 architecture and limitations

Figure 2 shows v1 (`main.py`), our first production architecture. It validated that relative text and visual evidence are both necessary, but ranking policy was **fragmented**: heuristic sort, then LLM hard rerank, Hit@1 guard, deterministic promote, diagnosis promote, VLM ensemble, and post-blend guard.

Table 2: Benchmark fault-type frequencies (n=22 traces). Coverage is collector-driven (e.g., seven HTTP-error traces, one JS exception).

Failure type	n
HTTP 404/500	7
Wrong navigation / invalid input	5
UI state mismatch	3
Timeout / connection lost	2
Auth-gated / credential failure	2
Backend OK, frontend fail	2
JS exception	1

Guards encoded patterns from individual benchmark traces (promote causal action on `hackernews`; lock strong verify-pixel on `youtube`; suppress spurious scroll on `amazon`). Tuning for one pattern regressed others; e.g., blocking deterministic promotes under visual lock sharply dropped accuracy on action-labeled traces in an intermediate experiment.

Hybrid mode (`-llm -vlm`) activated both override chains, so conflicts surfaced more often than in single-modality modes. On partial benchmark runs, v1 hybrid underperformed LLM-only despite each modality working well in isolation. Diagnosis and deterministic rules could also promote steps to rank 1 using patterns discovered while inspecting labeled traces, complicating claims of a general ranking policy.

v2 response: one fusion function, soft LLM/VLM priors, read-only diagnosis, structural caps replacing per-trace guard exceptions. v1 remains in the repository as a reference implementation; all reported main results use v2.

4 EVALUATION

4.1 Research questions

RQ1: How much do LLM, VLM, and hybrid fusion improve relative step ranking over the heuristic baseline?

RQ2: What is the contribution of pixel/visual channels (no-pixel ablation)?

RQ3: How do LLM-only and hybrid compare across Hit@k and rank-quality metrics?

RQ4: How does performance vary across data sources and fault signal types?

4.2 Benchmark and setup

We evaluate on **22 traces** from three independent collections: 8 from efe/irem, 10 from areeb/salem, and 4 from ersel (reported as a *test set* in figures due to distinct schema and intent fields). Ground truth specifies one fault step per trace. Table 2 summarizes fault-type frequencies; the full trace catalog is in our repository.

All main results use v2 (`main2.py`) with frozen run JSONs listed in `scripts/metrics_manifest.yaml`. Baseline: `python main2.py`. **Models (free tier).** LLM: `nvidia/nemotron-3-super-120b-a12b:free` via OpenRouter (T=0). VLM: `meta-llama/llama-4-scout-17b-16e-instruct`, via Groq (T=0), five screenshot pairs per trace (`top_k_for_vlm`, Section 3.4). We chose separate free-tier providers to stay within daily quotas without paid credits; this constrains model choice and throughput relative to commercial APIs.

Table 3: v2 aggregate results on 22 traces. Baseline = heuristic with pixel. VLM n=21 (`dictionary` excluded: incomplete screenshot pairs for top-K candidates).

Mode	n	Hit@1	Hit@3	Hit@5	MRD
Heuristic	22	64%	91%	91%	0.86
No pixel	22	68%	86%	91%	0.86
LLM	22	**86%**	95%	100%	0.32
VLM	21	67%	95%	100%	0.43
Hybrid	22	82%	**100%**	100%	**0.27**

MRD = mean rank distance. MAD@5: Heuristic 7.94, LLM 6.94, Hybrid 6.07.

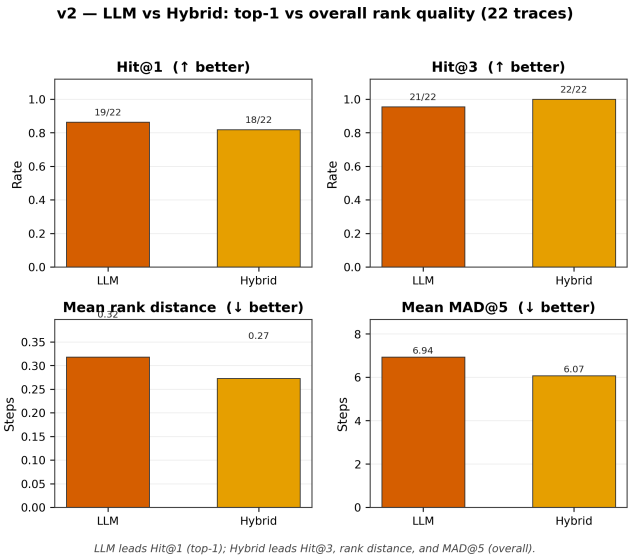

Figure 3: LLM vs. hybrid (v2, 22 traces). LLM leads on Hit@1 (top-1 localization); hybrid leads on Hit@3, mean rank distance, and MAD@5 (overall rank quality).

4.3 Results

Table 3 reports aggregate v2 metrics on all 22 traces.

RQ1. LLM improves Hit@1 from 64% to 86% (+22 points); hybrid reaches 82% (+18). Every mode except no-pixel heuristic beats the baseline on Hit@3 or rank distance.

RQ2. Removing pixel changes Hit@1 from 64% to 68% on this corpus; marginal on aggregate, but pixel and visual-causal channels are essential for screenshot-primary faults (Section 4.4).

RQ3. Figures 3 and 5 compare LLM and hybrid directly. **LLM achieves the highest top-1 rate** (86% Hit@1; 19/22 traces). **Hybrid is stronger on overall ranking quality**: 100% Hit@3 (vs. 95%), lower mean rank distance (0.27 vs. 0.32), and lower MAD@5 (6.07 vs. 6.94). Neither mode dominates every metric; per-trace disagreements (e.g., `amazon`, `github`) are analyzed in Section 4.4.

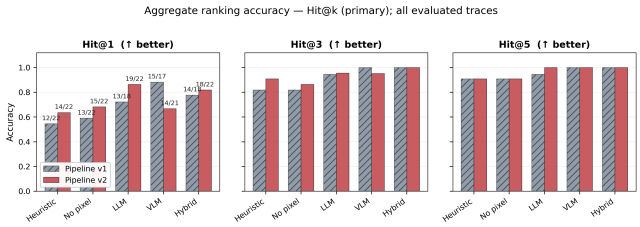

Figure 4: Hit@1/3/5 across modes; v1 vs. v2 evolution. Heuristic is the baseline.

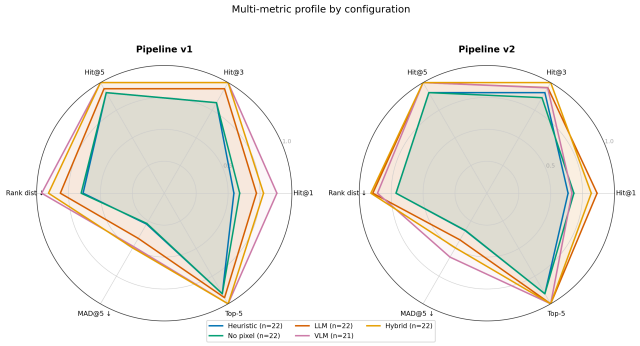

Figure 5: Multi-metric radar (v1 and v2). On the v2 panel, LLM extends furthest on Hit@1; hybrid leads on Hit@3/5, rank distance, and MAD@5.

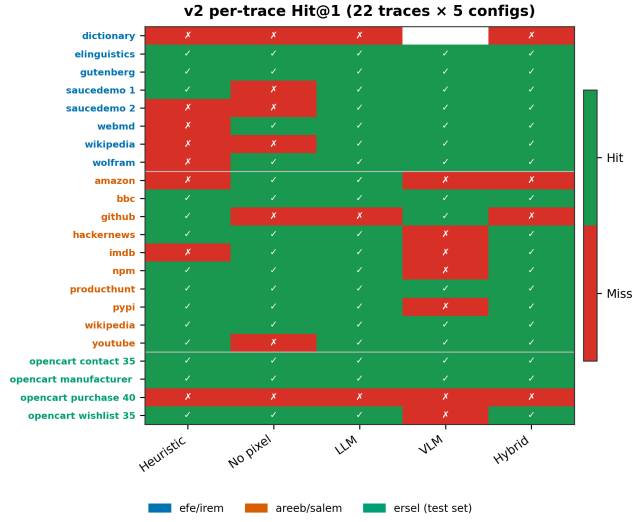

Figure 6: Per-trace Hit@1 heatmap (v2). Trace names colored by source (legend). Blank VLM cell: `dictionary` excluded (missing screenshots for top-K steps).

Figure 4 shows Hit@k for all five configurations; Figure 5 summarizes multi-metric profiles. Figure 6 gives per-trace Hit@1; Figure 7 breaks Hit@1 by source collection.

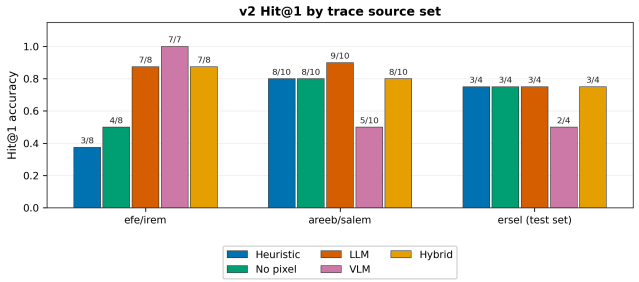

Figure 7: v2 Hit@1 by source collection and mode.

4.4 Case studies

amazon (GT step 20, auth-gated action). LLM ranks the causal action first (Hit@1). Hybrid prefers an earlier scroll step with stronger pixel signal (rank distance 2). Illustrates visual noise overriding causal text.

github (GT step 23, JS crash). Both LLM and hybrid miss Hit@1, but hybrid places GT at rank 2 (distance 1) vs. LLM rank 5 (distance 4). Hybrid recovers a visually evident but text-ambiguous fault in the shortlist.

saucedemo_2, bbc, pypi, youtube (screenshot-primary). The heuristic baseline misses `saucedemo_2` (no text signal); LLM and hybrid both localize it using local visual-causal and pixel channels in v2.

4.5 Analysis by signal type

Grouping traces by primary signal type reveals expected mode strengths. **Network/console faults** (9 traces): LLM reaches 89% Hit@1 (8/9) vs. 67% (6/9) for the heuristic baseline. **Screenshot-primary faults** (4 traces): the baseline reaches 75% Hit@1 (3/4); both LLM and hybrid reach 100% because v2's local visual-causal scan runs even in LLM-only mode. **Absence-of-activity** (`wolfram`, both `wikipedia` traces): navigation and missing-page features are critical; LLM reranking promotes the silent navigation fault over CDN noise.

4.6 Discussion

Table 3 and Figures 3–5 show that no configuration dominates every metric. The heuristic baseline (64% Hit@1) is fast and API-free but misses causal and screenshot-primary faults. **LLM-only** achieves the highest top-1 rate (86% Hit@1) and suits workflows that need a single best suspect. **Hybrid** is strongest on overall ranking quality (100% Hit@3, lowest mean rank distance and MAD@5) and suits workflows that need a robust shortlist. **VLM-only** is competitive on top-k but weaker alone at top-1; we treat it primarily as a hybrid component given API cost. **No-pixel** ablation shows marginal aggregate gain from local screenshot diff but clear value on screenshot-primary traces (Section 4.4).

4.7 v1 comparison

On partial runs, v1 heuristic reached 55% Hit@1 (n=22); v1 hybrid ~70% (n=10) did not reliably beat v1 LLM (~72%, n=18) due to guard

interactions. v2 hybrid on the full corpus reaches 82% Hit@1 with 100% Hit@3 (Table 4 in the appendix).

5 THREATS TO VALIDITY

Construct validity. Single fault-step labels simplify evaluation but blur symptom vs. cause (e.g., `amazon`). Hit@k and rank distance partially mitigate this by rewarding near-misses.

Internal validity. Fusion weights were fixed in `v2/config.yaml` before aggregate reporting. Structural rules were motivated by recurring failure *types*, not by per-trace grid search on ground truth in v2.

External validity. 22 English UI traces from academic collectors are not industry-scale. Three trace schemas limit generalization without adapter code. Table 2 shows uneven fault-type coverage, so mode strengths on rare categories (e.g., JS exceptions, n=1) are estimated from small samples.

Conclusion validity. LLM/VLM APIs with T=0 may still vary by provider routing; we report frozen run timestamps in the repository. Our evaluation used **free-tier** endpoints only: Nemotron 3 Super 120B on OpenRouter for text reranking and Llama 4 Scout 17B on Groq for screenshot pairs. Free tiers impose daily request caps, TPM throttling, and a restricted model catalog—paid alternatives (e.g., Qwen2.5-VL-72B, Gemini 2.0 Flash) may rank differently. Because the LLM and VLM run on *different* providers and model families, we do not assume equal capability across modalities; we report their complementary contributions under a fixed, cost-bounded configuration.

Evaluation bias. ersel (4 traces) uses intent fields absent elsewhere. VLM-only aggregate excludes `dictionary` (n=21): the trace artifact lacks complete pass/fail screenshot files for the heuristic top-K candidates (~5 steps), so the VLM could not analyze the ranked steps and the run is treated as unavailable (not scored as a miss). Hybrid mode still reports a fallback ranking for `dictionary` using text and pixel channels.

6 CONCLUSION

Relative debugging of UI test regressions benefits from multimodal pass–fail comparison. TraceLens demonstrates a reproducible pipeline from aligned trace diffs through LLM/VLM priors to ranked suspects and stakeholder explanations. Against a 64% heuristic baseline, results show LLM-only achieves the best top-1 localization (86% Hit@1) while hybrid fusion is strongest overall (100% Hit@3, lowest rank distance and MAD@5). v2's unified fusion replaces v1's trace-specific guard brittleness with a single interpretable scorer.

Future work includes a larger public relative-debugging benchmark, learned fusion weights, multi-fault labels, CI integration, and practitioner user studies.

DATA AVAILABILITY

The repository includes trace data, ground truth, the v2 pipeline (`main2.py`), frozen metric JSONs, and figure scripts. Regenerate figures with `python scripts/plot_metrics.py`.

Table 4: v1 vs. v2 Hit@1 on overlapping runs.

Mode	v1 Hit@1	v2 Hit@1
Heuristic	55% (n=22)	64% (n=22)
LLM	72% (n=18)	86% (n=22)
VLM	88% (n=17)	67% (n=21)
Hybrid	70% (n=10)	82% (n=22)

A V1 AGGREGATE COMPARISON

v1 VLM headline accuracy on 17 traces should not be compared directly to v2 VLM on 21 traces; different coverage and fusion integration dominate.

REFERENCES

[1] David Abramson, Ian Foster, John Michalakes, and Rok Sosiç. 1995. Relative Debugging and its Application to the Development of Large Numerical Models. In *Proceedings of the 1995 ACM/IEEE Conference on Supercomputing (SC)*. San Diego, CA, USA, 51.

[2] David Abramson, Ian Foster, John Michalakes, and Rok Sosiç. 1996. Relative Debugging: A New Paradigm for Debugging Scientific Applications. *Commun. ACM* 39, 11 (1996), 67–77. https://doi.org/10.1145/240455.240475

[3] Hiralal Agrawal, Joseph R. Horgan, and Si-Leung Wong. 1995. Fault Localization Using Execution Slices and Dataflow Tests. *IEEE Transactions on Software Engineering* 21, 2 (1995), 149–162. https://doi.org/10.1109/32.345428

[4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 1877–1901.

[5] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[6] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* (2024). https://doi.org/10.1145/3695988

[7] James A. Jones, Mary Jean Harrold, and John Stasko. 2002. Visualization of Test Information to Assist Fault Localization. In *Proceedings of the 24th International Conference on Software Engineering (ICSE)*. 467–477. https://doi.org/10.1145/1060527.1060605

[8] Ali Mesbah, Arie van Deursen, and Stefan Lenselink. 2012. Crawling Ajax-Based Web Applications through Dynamic Analysis of User Interface State Changes. *ACM Transactions on the Web* 6, 1 (2012), 3:1–3:30. https://doi.org/10.1145/2076796.2076799

[9] Filippo Ricca and Paolo Tonella. 2001. Analysis and Testing of Web Applications. In *Proceedings of the 23rd International Conference on Software Engineering (ICSE)*. 25–34. https://doi.org/10.1145/381896.381900

[10] Rok Sosiç and David Abramson. 1997. GUARD: A Relative Debugger. *Software: Practice and Experience* 27, 2 (1997), 185–206. https://doi.org/10.1002/(SICI)1097-024X(199702)27:2<185::AID-SPE104>3.0.CO;2-1

[11] Ming Wen, Junjie Chen, Yongqiang Tian, Rongxin Wu, Dan Hao, Shi Han, and Shing-Chi Cheung. 2021. Historical Spectrum Based Fault Localization. *IEEE Transactions on Software Engineering* 47, 11 (2021), 2348–2368. https://doi.org/10.1109/TSE.2019.2948158 Journal-first version, ICSE 2020.

[12] W. Eric Wong, Ruizhi Dong, Yu Qi, and Jian Zhang. 2016. A Survey on Software Fault Localization. *IEEE Transactions on Reliability* 65, 3 (2016), 841–867. https://doi.org/10.1109/TR.2016.2527818

[13] Wei Xu, Ling Huang, Armando Fox, David A. Patterson, and Michael I. Jordan. 2009. Detecting Large-Scale System Problems by Mining Console Logs. In *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 117–132.

[14] Andreas Zeller. 2002. Simplifying and Isolating Failure-Inducing Input. *IEEE Transactions on Software Engineering* 28, 2 (2002), 183–200. https://doi.org/10.1109/32.988498

[15] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. GPT-4V(ision) is a Generalist Web Agent, if Grounded. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, Vol. 235. 61349–61385.

[16] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations (ICLR)*.